

Planet Pawsitive - Team31

Planet Pawsitive - Team31 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games (*gamified-app lens*) · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

Proposal (as submitted): *Planet Pawsitive* encourages Singaporeans to build a lasting habit of using the **ReturnRight** reverse-vending machines. It has a **location-based tracker** (nearest machine) and a **Tamagotchi mini-game**: each bottle deposited grants a **token** (used for feeding / drawing new tamagotchi), with the token amount determined by an **uploaded image of the machine's return summary**. The **pet's state doubles as the reminder system** (prompting returns without scheduled notifications); decorations + interactable objects for the pet's space are a **stretch goal**. The main interface reflects ongoing activity and surfaces context on **Singapore's waste situation**, and a **lifetime tracker** records total containers returned + deposits recovered. Named load-bearing C++ (local engine): the **real-time interactive pet simulator** + pet progression, the **real-time pet growing** synced to a **world clock** driving a **day/evening/night room** cycle, the **custom Pet AI** (random actions — sleeping/walking/sitting), and **custom 2D technical art** (light source, shaders) — all in C++. Named technical features: **OCR image recognition** (in Python) on the machine's confirmation screen, with a **timer to prevent fraudulent use**; the real-time pet growing + day/night world-clock sync; the custom Pet AI; and the custom 2D technical art (lighting/shaders). Tools: the **OCR scan** of return-summary images; and an **accessories/decorations editor**. Design features: the **Tamagotchi pet system** (emotional-responsibility lifestyle framing); the **scan-to-track mechanic** (photo → OCR reads container count → tokens); a **habit-forming loop** (streaks, cumulative bottles, pet-status nudges instead of notifications); and a **"Semakau" clock** (a Singapore-landfill countdown — "Your 5 bottles have stayed out of Semakau island!" / a countdown to 2035 — inspired by the Climate Clock). Team: **6 CS + 1 Design + 0 Art**.

Note (category): like other **gamified apps** (there is no sustainability/health category), Planet Pawsitive is filed under **Games** and assessed through the

gamified-app lens (pet/progression/habit-loop systems) — the same way the module treats step-and-pet and habit-loop projects. The recycling/awareness is the theme; the pet-habit loop is the product.

Note (module completeness): the app needs a **backend + external DB** (pet state, tokens, lifetime tracker, OCR results) — use Firebase and/or the team's Linux server (set the seam up + basic-test it by M2). *Platform:* location tracker + image upload → a **mobile** build (Native Android, Pixel 9a). *On the Python OCR:* the module's graded core is **C++** — the pet sim / AI / day-night / technical art are correctly in C++; the **Python OCR is fine as a backend/tool** (like a backend script), but keep the load-bearing real-time logic in C++ and treat OCR as a service the app calls (with the anti-fraud timer server-checked). *Monetization:* none specified and the economy is **earn-by-recycling** (tokens), which is clean.

References:

- [Recycle N Save — Smart Reverse Vending Machines \(Singapore\)](#) (the real Singapore scheme this app is built on: deposit drink containers at reverse-vending machines and earn rewards — the exact scan-a-deposit-earn-a-reward + Singapore-recycling context)
- [Tamagotchi — Wikipedia](#) (the archetypal **virtual pet** you raise through life stages, where the **pet's needs periodically remind** you to care for it — the pet-as-reminder + care/growth precedent)
- [Climate Clock — Wikipedia](#) (the countdown clock showing the time left to limit warming to 1.5°C — the team's stated inspiration for the "Semakau" landfill-countdown clock)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Planet Pawsitive
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ pet engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — location, camera/image upload and on-device state are native concerns. The Native PC build is the dev/tools vehicle (the decorations editor + the OCR pipeline). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for pet/token/tracker persistence and the OCR/anti-fraud service (Python OCR can run here).

Front end: A 2D app — the pet + its room with day/night lighting (custom technical art), the token/feeding/drawing UI, the machine-locator map, the scan-upload flow, the lifetime tracker and the Semakau clock. **Languages:** Native — Kotlin Native Android UI + the C++ pet engine via JNI (Android Studio); OpenGL ES for the realtime pet/room + shaders in a GLSurfaceView; tracker/map/menus can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The local C++ engine — the real-time pet simulator + progression, the real-time growing synced to a world clock, the day/evening/night room cycle, the custom Pet AI (random actions) and the custom 2D technical art (lighting/shaders) — plus a backend (Firebase/Linux server) for persistence and the Python OCR + anti-fraud-timer service that converts a return-summary image into tokens. **Languages:** C++ (pet engine — load-bearing); Python OCR on the backend/tool tier; Firebase on the Extended tier.

Backend scope: Local now, server-backed (T2) — the persistence + OCR/anti-fraud seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ engine is the heart — it runs the real-time pet simulator, grows the pet against a world clock, drives the day/evening/night room cycle, runs the custom Pet AI (sleeping/walking/sitting), and generates the custom 2D technical art (lighting/shaders).
- *Data-driven:* pet states + growth stages, token costs/actions, decorations, the Semakau-clock facts and the lifetime tracker are all data — a new pet action, decoration or token cost is a data change (authored in the decorations editor).
- *2D:* the pet + room, the custom lighting/shaders, the map, the scan flow, the tracker and the Semakau clock (custom-rendered) satisfy the 2D gate. *3D:* not needed.
- *Front-end + Back-end:* a client/server split — device app + backend for persistence + the OCR/anti-fraud service; the scan-upload → token flow is **Application Communication ✓**.
- *Custom tools:* two — the **OCR scan** pipeline (Python, with the anti-fraud timer) and the **accessories/decorations editor**.
- *Networking:* single-player with async backend, so a real-time Networking System is optional (●). *External DB:* pet/tokens/tracker/OCR results

(Firebase — add per the module). *Auth* » / *Analytics* »: module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the pet sim + real-time growing + world-clock/day-night cycle.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a real-time C++ loop running the pet simulator, growing the pet against a world clock, and driving the day/evening/night room cycle; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the Pet AI (M1G8) and technical art (M1G4) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** A data model with pet states + growth stages, token costs/actions, decorations, Semakau-clock facts and the lifetime tracker in hot-reloadable JSON — a new pet action, decoration or token cost is a data change.
- **Prerequisites:** G1(loop & entity hooks).
- **Feeds into:** M2G2 (the decorations editor), M2G13 (token/habit systems), M1G8 (Pet AI data).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — incl. OCR-pipeline testing.

Tech & dependencies

- **Tech:** ImGui overlays (pet AI state, growth/world-clock, token grants), Tracy profiling of the pet sim + shader/lighting, Catch2 tests (growth/day-night logic, token conversion, anti-fraud timer, data loading), and OCR-pipeline test fixtures, on a GitHub Actions matrix.
- **Prerequisites:** G1 (loop to instrument).

- **Feeds into:** M2G3 (CI hardening); the scan-to-token verification.
- **G4 2D Graphics** — the **custom 2D technical art** (light source, shaders) + the pet/room + day/night.

Tech & dependencies

- **Tech: Web build:** A renderer with custom lighting (a light source) and shaders for the pet + room, day/evening/night tinting, and the token/feeding UI; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (advanced lighting/shaders); the pet's visual charm.
- **G8 Enemy AI & Pathfinding** — the **custom Pet AI** (random actions: sleeping/walking/sitting).

Tech & dependencies

- **Tech:** A behaviour/decision system (weighted-random or utility) choosing pet actions (sleep/walk/sit/idle) with simple in-room pathing, influenced by pet state (hunger/happiness) and the time of day; custom C++.
- **Prerequisites:** G1 (tick), G2 (pet data).
- **Feeds into:** M2G13 (pet progression); the pet's believable liveliness.
- **G12 Save / Persistence & Firebase Integration** — the lifetime tracker + tokens + state + the OCR scan-to-token backend.

Tech & dependencies

- **Tech:** Local persistence of pet state, tokens and the lifetime tracker, synced to Firebase; and the scan flow — the return-summary image goes to the OCR service (Python), which reads the container count and (with the anti-fraud timer, server-checked) grants tokens.
- **Prerequisites:** G2 (data model).
- **Feeds into:** The scan-to-track mechanic; M2 habit systems.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — on-device perf + the backend seam.

Tech & dependencies

- **Tech: Web build:** Memory pools, Tracy profiling of the pet sim + shaders, ASan on MSVC, and on-device validation (Pixel 9a) — including the pet running efficiently in the background for the real-time growth; the Firebase/Linux-server seam (persistence + OCR/anti-fraud) set up (account + software + basic connectivity test) by M2.
- **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a battery-viable real-time pet.
- **G2 ★ Content Editor — Core (Accessories/Decorations Editor) —** author decorations + interactable objects.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: place/author decorations and interactable objects in the pet's space, set token costs, and round-trip JSON, with live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
- **Prerequisites:** G2 (decoration data model).
- **Feeds into:** The decorations stretch goal; the custom-tools showcase.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (growth/day-night, token conversion, anti-fraud, OCR-pipeline fixtures, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.
- **G4 2D Graphics — Advanced (Lighting & Shaders) —** the custom technical art matured.

Tech & dependencies

- **Tech: Web build:** OpenGL/GLSL shaders, a proper light source + soft shadows, day/evening/night post-FX and atmosphere in the pet's room, texture atlases. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** M1G4 (2D graphics), M2G1 (optimization).
 - **Feeds into:** The signature "custom 2D technical art"; the day/night mood at showcase.
- **G6 Content Editor — Advanced Tool Features (Interactable Objects)** — the decorations editor matured.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets: author interactable objects (tap responses, pet interactions) and richer room layouts with live preview binding, drag-drop and deeper undo/redo.
 - **Prerequisites:** M2G2 (editor core).
 - **Feeds into:** The interactable-objects stretch; the content showcase.
- **G13 Gameplay Systems II — the token economy + habit loop + Semakau clock + lifetime tracker.**

Tech & dependencies

- **Tech:** C++ systems over the data model: the token earn/spend economy (feed/draw), the pet-progression + pet-state-as-reminder nudge system, the habit loop (streaks, cumulative bottles, optional money-saved), the Semakau clock (facts + countdown) and the lifetime tracker; balancing-data-driven.
- **Prerequisites:** M1G2 (data), M1G12 (tokens/tracker).
- **Feeds into:** The habit/awareness showcase; T2 content.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + **1 Art** (8) — the proposed **6 CS + 1 Design + 0 Art** is the module's tight case for an **art-forward** concept: a Tamagotchi pet with **growth stages, day/night rooms, decorations and drawable/collectable tamagotchi** is a real art surface, so with no artist the single designer must also carry the **D11 conditional** each milestone, and a **solo designer + 0 artists** is exactly what the design pool flags ("must ask the instructor which conditional bends", see [design-goals-pool.pdf](#)). *Mitigating factor:* the proposal's **custom 2D technical art (lighting/shaders) is CS-generated**, which offloads some of the visual polish onto the engine — but the pet, room and decoration **assets** still need an artist's hand. **Recommended: add 1 Artist** (best for the pet/room/decoration charm), or at least a **2nd designer** so one owns the habit/economy design and the other absorbs the D11 art surface. If the team stays 0-artist, scope hard: **one pet with a few growth stages + a small reusable decoration set** authored under D11, and expect the instructor sign-off on the bent conditional. *Scope note (advisory):* a real-time pet engine + custom Pet AI + shader/lighting tech art + an OCR scan pipeline + a decorations editor is a solid scope — the pet sim + world-clock growth + day/night + the OCR-to-token scan + one decoration is the M1/M2 floor; the advanced lighting/shaders, the interactable-objects editor and the full Semakau-clock + habit-loop polish are the stretch. **OCR reliability + anti-fraud** is the trickiest bit — validate the scan pipeline early with real machine-summary photos, and check the anti-fraud timer server-side.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track as proposed — 0 artists, so the pet/room/decoration Art surface is carried by the Design-owned D11 conditional; adding an artist is recommended, see above); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the recycling-habit + Tamagotchi USP, the scan-to-track mechanic, the pet-state-as-reminder loop and the Semakau clock; the problem — sustaining recycling habits; a primary benchmark e.g. Recycle N Save / a virtual-pet app, and the Singapore/Semakau local-awareness angle)

- M1D9 Engagement & Retention Loop Design (the habit-forming loop — streaks, cumulative bottles, pet-status nudges instead of notifications — with the ethics line: engagement serves the recycling habit, never dark-pattern)
- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the pet visual direction (growth stages), the day/night room look, the decoration set and UI style; 2 mockups.

• **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — the pet, day/night room, decorations and UI implemented in-engine, no placeholders.

• **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the pet/room, the scan-upload flow, the machine-locator map, the token/feeding UI and the Semakau clock, plus a wireframe baseline.

• **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the pet space, scan flow, tracker, map and Semakau clock navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished scan-to-pet + habit UX tested with real users on the Pixel 9a (does the pet-as-reminder actually drive returns?).

• **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working in the prototype (calm ambient BGM by time-of-day, pet actions, feed/draw, token-earned and scan-success cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable playback (day/evening/night ambience).

• **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the app's audio set (time-of-day ambience + pet / feed / token / scan / milestone cues, ≥6 types), suited to a calm lifestyle app.

- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback the build needs (multi-channel mixing; distinct scan-success + token cues; day/night ambience crossfade).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the OCR scan pipeline + the decorations editor), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*